

Natural Language Processing - Project

Part 1.A

Part 1.A

In this, you have to modify the baseline GPT2 by adding a set of techniques that might improve the performance. In this, you have to **add one modification at a time incrementally**. If adding a modification decreases the performance, you can remove it and move forward with the others. However, in the report, you have to provide and comment on this unsuccessful experiment. For each of your experiments, you have to print the performance expressed with Perplexity (PPL).

One of the important tasks of training a neural network is hyperparameter optimization. Thus, you have to play with the hyperparameters to minimise the PPL and thus print the results achieved with the best configuration (in particular **the learning rate**).

Mandatory requirements: For the following experiments the perplexity must be below 250 (**PPL < 250**).

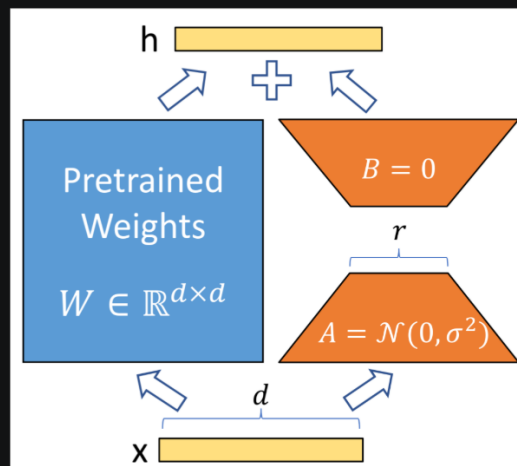
0. Baseline: find a suitable learning rate, but keep the other hyperparameters fixed.
1. Hyperparameter optimization: incrementally change hyperparameters one at a time (`d_model`, `n_heads`, `num_layers`, `ff_dim`). You may have to readjust the learning rate for larger model sizes.
2. Add dropout layers: --> [link](#).
 - one after obtaining the embeddings (after summing token embeddings with positional embeddings)
 - after computing the attention weights
 - after the output projection in `MultiHeadAttention`
 - after the last linear layer of the Feed Forward
3. Add weight tying: the input embeddings (`token_embed`) and the output layer (`lm_head`) have to share the exact same weights

Part 1.B

Part 1.B

While we can train GPT2 from scratch, we may benefit from the pre-trained weights. However, as you might have noticed from the previous part, training large models requires a large amount of resources, and training the full-sized GPT2 (small) may require more resources than are available to us.

How can we solve this? Instead of fine-tuning all parameters, we can train a small fraction of additional parameters called adapters. The most popular and widely used technique is Low Rank Adaptation (LoRA), presented in [this paper](#).



Starting from the pre-trained model from Huggingface, fine-tune GPT2 using LoRA (Low Rank Adaptation). **You have to implement LoRA yourself manually**, libraries such as PEFT are not allowed. Complete the `GPT_LoRA` model we provide below by adding the LoRA adapters and only make these adapters trainable.

- Implement LoRA and fine-tune GPT2 by training only the LoRA adapters. Apply the LoRA adapters to the query, key, and value weight matrices.
- Experiment with the `rank` and `alpha` hyperparameters.

Mandatory requirements: in the experiments with LoRA the perplexity must be below 250 (**PPL < 250**) and it should be lower than that achieved in Part 1.A.